Leveling Module 1: Applied ML Workflow Mini-Project

CS640: Applied Artificial Intelligence and Machine Learning – completed worksheet

Field	Student Response
Student Name	[enter your name]
Date	November 8, 2026
Dataset / Task Name	SECOM semiconductor manufacturing data – detecting failing wafers from process-sensor readings (binary fault detection)
Notebook / Repository Link	[paste your link]. Local files: SECOM_LR_vs_RF_v2.ipynb (experiments E1–E9, main evidence), SECOM_Best_Model_v3.ipynb (leak-free re-check, nested tuning, drift study), results/scoreboard.csv, results_v3/*.csv

4. Dataset and Task Description

Prompt	Student Response
Dataset source or URL	UCI Machine Learning Repository, SECOM dataset (https://archive.ics.uci.edu/dataset/179/secom). Files used: secom.data (sensor matrix) and secom_labels.data (label + timestamp).
Dataset context: who collected it? What does it represent?	Donated to UCI by McCann & Johnson (2008). Each row is one wafer/production entity from a semiconductor fabrication line, described by 590 process-monitoring sensor readings, with a pass/fail result from in-house line testing and a timestamp. Data cover 19 Jul to 17 Oct 2008.
Number of instances	1,567 wafers: 1,463 pass (93.4%) and 104 fail (6.6%), about 14 passes per failure.
Input features or inputs	590 numeric sensor readings per wafer. After label-free cleaning (116 constant sensors and 32 sensors missing in more than 40% of wafers removed) 442 sensors remain. The timestamp is used only to order the data and for the time-ordered check, never as a model input.
Target/output variable	Binary label: fail (+1 in the file, coded 1) vs pass (-1, coded 0). The model outputs a calibrated failure risk per wafer.
Task type	Binary classification (rare-event fault detection). SECOM has no fault-type labels.
Operational context	A process or yield engineer who must decide which wafers to flag for extra inspection or re-test before they move on. A missed failure is assumed to cost 10 times a false alarm (a 10:1 cost assumption I set; it is not given by the dataset), so a wafer is flagged when its predicted risk is at least 1/11 = 0.091.
Known dataset issues	(1) Strong imbalance, 6.6% failures, so accuracy is meaningless (always-pass scores 93%). (2) Missing values: 538 of 590 sensors have gaps, 32 are missing in more than 40% of wafers. (3) 116 constant sensors. (4) Scales span ~8 orders of magnitude, 247 of 442 sensors strongly skewed, readings up to 40 SD from the mean. (5) Heavy redundancy: 303 of 442 sensors have a near-copy ($\|r\| > 0.9$). (6) Weak signal: median single-sensor AUC 0.524, best 0.692. (7) Drift: failure rate falls from 22% (Jul) to 9% (Aug), 3% (Sep), 6% (Oct), and strong sensors drift with time (time order alone gives AUC 0.636). (8) Small sample of failures (104). No duplicate rows and no duplicate sensors.

6. Baseline and Validation Plan

Prompt	Student Response
Baseline model/system selected	E1: majority-class predictor (always "pass"), the floor. E3: L2 logistic regression on quantile-transformed sensors, the first real model and the reference point for the improvement. (E2, the same model on standardized sensors, was a pre-step.)
Why this is an appropriate baseline	Both are listed in the baseline taxonomy for classification, are cheap and reproducible, and are easy to beat. The majority baseline shows what "doing nothing" costs (0.664 per wafer); logistic regression shows how much a simple linear model on the sensors adds.

Prompt	Student Response
Validation strategy	Stratified 5-fold cross-validation repeated 5 times (25 held-out folds, identical folds for every experiment, seed 42). Everything that learns from data (imputation, scaling/quantile transform, sensor selection, hyper-parameters, probability calibration) is fitted inside the training fold only. Because the data are time-ordered and drift, every experiment also gets a one-shot time-ordered check: train on the earliest 80% of wafers (1,254; 87 fails), test on the latest 20% (313; 17 fails). The v3 notebook adds nested tuning, forward-chaining CV (train on earlier blocks, test on the next) and a shuffled-label sanity check.
Primary metric	Expected cost per wafer = (10 x missed failures + 1 x false alarms) / wafers, at the fixed risk threshold 1/11.
Why this metric is appropriate	Missing a failing wafer is much worse than a false alarm, and with 6.6% failures accuracy is useless. Cost per wafer encodes that trade-off in one number. The 10:1 ratio is an assumption, so ranking quality is reported separately.
Secondary metric(s)	ROC-AUC, PR-AUC (base rate 0.066), balanced accuracy, fail-class recall and precision, flag rate, log-loss/Brier score and calibration, plus late-wafer cost and ROC-AUC. Weighted precision is deliberately not used: the always-pass baseline scores 0.87 on it.
What counts as meaningful improvement?	Decision rule fixed before running: keep a change only if it lowers mean cost versus its parent experiment AND is cheaper in at least 15 of 25 folds. Fold-to-fold SD of cost is 0.05–0.07, so a mean change smaller than about 0.03 (|change / SE| below ~2) is treated as noise, and a change that wins in random CV but fails the time-ordered check is flagged rather than celebrated.

7. Improvement Attempt

Prompt	Student Response
Improvement attempted	Model change (E3 to E4): replace the logistic regression with a calibrated random forest (200 trees, median imputation, no scaling; leaf size tuned and the vote share turned into a probability with a Platt step, both on out-of-bag predictions). Follow-ups tested the same way: feature selection (E5), tuned max_features (E6), a LR + forest ensemble (E7), detrending (E8), gradient boosting (E9). See the log in Section 10.
Hypothesis	Failures depend on combinations of sensors and on extreme values. Trees ignore scale and outliers and can model interactions, so the forest should rank wafers better than a linear model and lower the cost at the same 1/11 threshold.
Result compared to baseline	Same 25 folds, same metric. Majority baseline cost 0.664; E3 (LR) 0.537 (cheaper than E1 in 24 of 25 folds); E4 (forest) 0.510, i.e. -0.026 vs E3, cheaper in 18 of 25 folds, change / SE about -2.0. ROC-AUC 0.742 vs 0.695, PR-AUC 0.189 vs 0.180, balanced accuracy 0.646 vs 0.633, precision 0.175 vs 0.149, recall 0.442 vs 0.450. Pooled over all wafers, E4 flags 266 wafers on average (46 true failures, 220 false alarms) against E3's 316 (47 true, 269 false), so the forest saves false alarms without catching more failures. **Time-ordered check: E4 is worse than E3** (late cost 0.588 vs 0.482, late ROC-AUC 0.549 vs 0.675), and 0.588 is even above the late baseline of 0.543. Averaging the two (E7a, fixed in advance) keeps E4's random-CV cost (0.511) and gets the best late cost of all experiments (0.470, ROC-AUC 0.643). The more thorough v3 re-run (nested tuning, LR + RF + LightGBM ensemble) reaches PR-AUC 0.219, ROC-AUC 0.746, cost 0.50, statistically indistinguishable from v2's best (corrected t about 1.6 vs LR).
Decision	**Keep E4 as a modest gain in random CV (borderline), but investigate further / do not trust it alone.** The cost gain is about 0.4 of a fold SD, the evidence is borderline, and it does not survive the time-ordered check. Recommended final choice: the pre-registered LR + forest average (E7a), or the v3 calibrated ensemble, retrained regularly on recent wafers. Rejected: detrending (E8) and gradient boosting (E9), and top-k selection for logistic regression (E5a).

Evidence standard met: every comparison uses the same 25 folds, the same cost metric and threshold as the baseline.

8. Leakage Checklist

Checklist item	Yes / No / N/A	Brief explanation
Did I split the data before fitting preprocessing steps such as scaling, encoding, feature selection, or PCA?	Yes (one caveat)	Imputation, quantile transform/standardizing, top-k selection, L1 selection, hyper-parameter tuning and Platt calibration are all fitted on the training fold only (selectors sit inside the inner CV). Caveat: dropping constant and mostly-missing sensors (442 kept) and the E8 detrending are done once on the whole data, but both use no labels, and no sensor has 17–46% missing, so every fold would drop the same ones. EDA statistics (single-sensor AUCs, correlations) used all rows but only to describe the data, not to choose features.
Are duplicate or near-duplicate records present across train/test or validation sets?	No exact; near-duplicates possible	No identical rows and no identical sensors. However, wafers made close together in time have similar readings, and random CV puts neighbours in both train and test, so random-CV scores are probably optimistic. This is why the time-ordered check and v3 forward-chaining were added.
Does any feature contain future information not available at prediction time?	No (by design)	Features are sensor readings of the same wafer. Timestamp is not a feature. The only time-based transform (E8 detrending) uses a causal rolling median of earlier wafers only, and it was rejected anyway. I cannot verify from the data documentation exactly when each sensor is read relative to the final test.
Is the target indirectly encoded in one or more features?	Not found, partly unverifiable	Missing-value count does not separate classes (6.3% vs 6.9% failure for wafers with/without gaps; mean NaN count 25.5 vs 26.9). The strongest sensors also drift with time, and failure rate depends on period (22% to 3%), so some sensors may act as a proxy for "when" rather than for the fault itself. A shuffled-label run in v3 gives chance results (no harness leakage).
For time-based data, did I preserve chronological order when appropriate?	Partly	Primary evaluation is random stratified CV (does not respect time). To compensate, every experiment has a train-early / test-late check, and v3 adds a 75/25 chronological hold-out and forward-chaining CV. These show a large drop: late ROC-AUC 0.52–0.69, hold-out near chance (0.51–0.56).
Did I avoid repeatedly tuning decisions on the final test set?	Mostly	There is no untouched final test set. Tuning is nested inside training folds, the threshold comes from the cost ratio (1/11) rather than the test fold, and the late-wafer check was run once per experiment. But I compared about 14 variants on the same 25 folds, so the best (E7b, cost 0.490) is optimistic; it is flagged as such. In v3, choosing the best of 15 ensembles on the same 10 folds is also mildly optimistic.
For document-based projects, could related chunks appear in both sets?	N/A	Not a document-based project.
Did I document any unresolved leakage concern?	Yes	Unresolved: (1) temporal neighbours across train/test in random CV; (2) drift-proxy sensors; (3) selection among many variants on shared folds; (4) the 10:1 cost is assumed. Net effect: random-CV results (ROC-AUC about 0.74) are an upper bound on how the model would do on future wafers.

9. Error Analysis

Model inspected: champion E7b (average of L1 logistic regression and tuned random forest), threshold 0.091, risk averaged over the 5 CV repeats so every wafer is scored by models that never saw it. Wafer ID = zero-based row in the chronological data file. Overall, 50 of 104 failures are missed (average risk 0.056) and 259 of 1,463 passes are flagged. Of the 104 failures, the top-10% highest-risk wafers contain 36 (recall 35%, precision 23% vs 6.6% base rate).

Case ID	Actual	Predicted (risk)	Error type / size	Possible cause	Possible fix
#1189 (30 Sep)	Fail	Pass (0.024; E3 0.035, E4 0.035)	False negative; risk 4x below the threshold, cost 10	Late-September wafer: failures are rare then (Sep rate 2.9%) and the models learned mainly from the fail-heavy Jul–Aug period. Hypothesis (not verified): nothing in its sensors looks abnormal to the models.	Retrain on recent wafers; add period-aware or process-step features; monitor drift.
#1438 (9 Oct)	Fail	Pass (0.025)	False negative, cost 10	Same pattern: a late wafer that looks ordinary to every model, so the signal for this failure is not in the strongest (drifting) sensors.	Forward-chaining retraining; look for failure types the models never saw.
#797 (11 Sep, near #795)	Fail	Pass (0.031)	False negative; #795 (0.030) failed two hours earlier and was missed too	Two consecutive failures with very low risk suggest a short fault episode or lot, and the model scores each wafer in isolation.	Add neighbour/lot-level features (rolling stats of recent wafers); alert on clusters.
#1458 (11 Oct)	Pass	Fail (0.625; E3 0.283, E4 0.270)	False positive; highest-risk wafer overall, cost 1	Hypothesis (not verified): an unusual sensor reading (extreme value) that both families score as risky, or the L1 model extrapolating. Both E3 and E4 also flag it, but only about 0.27 on their own.	Cap or cap-by-quantile more aggressively; manual check of its top sensors; calibrate the ensemble.
#275 (18 Aug) and #44 (29 Jul)	Pass	Fail (0.513 and 0.411)	False positives (high confidence)	Both are in the higher-failure early period (July: 22% failures, 67% of July wafers flagged). The model partly learned "early period = risky", so passing wafers from that time look suspicious.	Period-robust features or recency weighting; per-period thresholds.
#1328 (5 Oct)	Fail	Fail (0.129) but E3 0.191 vs E4 0.065	Caught only by the average; the forest alone would miss it	Linear and tree models disagree: 12 failures are caught only by LR, 12 only by the forest, 36 by both, 44 by neither. Different model families capture different failures.	Keep an ensemble rather than a single family (this motivated E7).
#45 (29 Jul)	Fail	Fail (0.153) but E3 0.085 vs E4 0.213	Caught only because of the forest; the LR sits just below the threshold	Opposite disagreement: an interaction/outlier the forest uses but the linear model underweights.	Same: average calibrated risks; try more diverse members.

Summary of failure pattern. Errors are not random: they are driven by time. The model is most confident, and most wrong, on early (July/August) wafers, where the failure rate is high and many passing wafers get flagged (July: 67% flagged, 22% actual failures; September: 4% flagged, 3% failures). Failures that occur in September and October are mostly missed (only 3 of 17 September failures are caught, 11 of 22 in October versus 11 of 14 in July), with risks near 0.02–0.05, so they look completely ordinary. The missed failures also lack the abnormal readings of caught ones (about 3 readings beyond 3 SD vs 7.5 for caught failures), which suggests the sensors carry little signal for those failures. Finally, linear and tree models each catch failures the other misses, which is why averaging helped.

10. Experiment Log

Cost = cost per wafer (lower is better) over 25 CV folds; "late" = train first 80%, test last 20% (always-pass late cost = 0.543). "Folds" = folds cheaper than the parent experiment.

Experiment	Change Made	Hypothesis	Result	Decision / Lesson
E1	Baseline: majority class (always pass)	Establish starting point	Cost 0.664, ROC-AUC 0.50, recall 0, balanced acc. 0.50	Floor. Anything real must beat 0.664.

Experiment	Change Made	Hypothesis	Result	Decision / Lesson
E2	L2 logistic regression, standardized sensors	Sensors carry enough signal to beat E1, but extreme readings (up to 40 SD) make risks overconfident	Cost 0.568 (cheaper than E1 in 25/25), ROC-AUC 0.687	Keep: sensors carry real signal.
E3	Quantile transform instead of standardizing (working baseline)	Bounding outliers and skew gives honest risks and lowers cost	Cost 0.537 (17/25 vs E2), ROC-AUC 0.695, late cost 0.482	Keep. Reference for later changes.
E4	Calibrated random forest (improvement)	Forest combines sensors and ignores scale, so it ranks better than E3	Cost 0.510 (18/25), ROC-AUC 0.742; late cost 0.588, late ROC-AUC 0.549	Keep, but borderline (about -2.0 SE) and fragile on late wafers.
E5a / E5b	Top-k sensor selection (selector inside the fold) for LR / forest	About 430 noise sensors hurt, so keeping top-k helps	E5a 0.541 (6/25), E5b 0.511 (11/25)	Reject: L2 already shrinks noise; k varied from 10 to all 442.
E5c	L1-penalized logistic regression	L1 picks one sensor from copy groups and generalises better	Cost 0.530 (12/25), ROC-AUC 0.704, recall 0.498	No clear gain on cost; best linear ranking. Used in E7b.
E6	Forest with tuned max_features and leaf size	Seeing more sensors per split finds the useful ones	Cost 0.503 (9/25), ROC-AUC 0.748, best single-model log-loss	No clear gain; larger max_features (0.3) won most often.
E7a / E7b	Average of calibrated risks: E3+E4 (fixed in advance) / E5c+E6 (chosen on same folds)	Linear and tree models make different errors, so averaging beats both	E7a 0.511, late cost 0.470 (best); E7b 0.490, 16/25, only -1.4 SE	E7a is the clean pick (robust on late wafers). E7b is the lowest CV cost but optimistic, since chosen on the same folds.
E8a / E8b / W100 / W400	Detrend each sensor by the rolling median of the previous W wafers (W = 200, plus 100 and 400)	Removing drift improves late-wafer results	Cost rose: LR 0.591, forest 0.550, W100 0.627, W400 0.580; late results no better	Reject, clearly. Slow sensor levels carry failure signal; shorter windows destroy more.
E9	Shallow gradient-boosted trees, calibrated	Boosting captures sparse interactions better than a forest	Cost 0.538, ROC-AUC 0.723, recall 0.354	Reject: too few failures (104) to learn from.
v3 check	Leak-free re-run: nested Optuna tuning, LR+RF+LightGBM ensemble, PR-AUC as main ranking metric, shuffled-label test, forward-chaining	A wider search and stricter protocol finds a clearly better model	PR-AUC 0.219 +/- 0.029, ROC-AUC 0.746, cost 0.50 (v2 best 0.49); shuffled labels at chance; hold-out ROC-AUC 0.51–0.56, forward-chaining about 0.65	Conclusion: practical ceiling is about ROC-AUC 0.75 / PR-AUC 0.22 on random CV. Drift is the real limitation, so retrain regularly.

Conclusion. What worked: the quantile transform (E3 over E2), a forest or a LR + forest average over a single linear model, and calibrating risks so the cost threshold means something. What failed: sensor selection, detrending and boosting. Every reasonable model lands in a narrow band (cost 0.49–0.54, ROC-AUC 0.69–0.75), differences among them are mostly within noise, and none holds up well on later wafers.

11. Team Capstone Connection

The main lesson is that the baseline and the evaluation design matter more than the model. For the capstone I would start with a majority/heuristic baseline plus a simple logistic regression, and decide the metric and decision rule up front: SECOM showed that accuracy is useless under imbalance and that an assumed cost ratio (10:1) with calibrated probabilities is more informative than a default 0.5 threshold, which flagged almost nothing (recall about 0.01). The biggest leakage risk I would watch for is time: random cross-validation looked good (ROC-AUC about 0.74), but the chronological check fell to near chance (0.51–0.56), so if the capstone data are time-ordered or grouped (users, documents, lots), I would split by time or group and put every fitted step inside the training fold. The error-analysis strategy transfers directly: aggregate metrics first, then look at high-confidence mistakes and group them by period or segment, which is how the drift pattern showed up. On feasibility, with only 104 positives most of my model changes were within noise, so I would budget time for better data and features and for retraining rather than for more model families. Based on this experience I could take the evaluation and experiment-tracking role (validation harness, leakage checks, experiment log, error analysis).

Evidence: scoreboard and per-experiment logs are in results/scoreboard.csv and results/experiment_log.csv (v2) and results_v3/.csv (v3); notebooks SECOM_LR_vs_RF_v2.ipynb and SECOM_Best_Model_v3.ipynb. Dataset: UCI SECOM (link in the header).*